
Tutorial 3 Solution

Computer Security BCS2420

Part A

1. **Correct Answer:** C. In offline attacks, the attacker can test millions of guesses per second without server interaction.
Explanation: Offline attacks do not require queries to the legitimate server. Attackers have the hashed password data and test guesses locally.
2. **Correct Answer:** B. Including salts in password hashing
Explanation: Salts ensure that each password hash is unique even if two users share the same password or if standard dictionary/rainbow tables exist.
3. **Correct Answer:** C. Slow down offline brute-force attacks
Explanation: Iterating a hash many times increases the computational work needed to verify each guess, slowing offline attempts significantly.
4. **Correct Answer:** B. Highly distinctive biometric features among the population
Explanation: If features are *very distinctive*, the system is less likely to mix up users. High distinctiveness usually **reduces** FAR, not increases it. The question asks which is *not* a contributor to increased FAR, so (B) is correct.
5. **Correct Answer:** B. Verify the entire chain by recomputing backward from the last OTP
Explanation: (h_0) is the final anchor; the server can verify each new OTP by stepping backward (or storing the next expected hash) in the Lamport chain.
6. **Correct Answer:** B. Users may get locked out or face increased login delays

Explanation: Rate-limiting extends or imposes delays after each failed attempt, which can inconvenience legitimate users if they forget or mistype passwords.

7. **Correct Answer:** C. Provide unique passwords for different sites without storing them all

Explanation: Derived-password systems combine a master secret with site-specific data to generate multiple distinct passwords.

8. **Correct Answer:** A. The overall user acceptance, because legitimate users may be unable to register

Explanation: Failure to Enroll (FTE) means some users can't even create a valid biometric template, reducing acceptance.

9. **Correct Answer:** C. It is kept hidden (not stored openly in the database), forcing attackers to guess it if the database is compromised

Explanation: A "pepper" is a secret salt. If not stored in plaintext with the hashes, attackers can't straightforwardly replicate the hash checks.

10. **Correct Answer:** A. A hardware token relying on an implicit time-based challenge

Explanation: The code changes every minute, implying a time-synchronized approach.

Part B

1. Offline vs. Online Attacks

- Technical Difference:

- **Online:** The attacker must submit guesses to the *legitimate* authentication server, receiving immediate feedback on correctness.
- **Offline:** The attacker has stolen or gained access to the hashed password file, so guesses can be tested locally without server involvement.

- Defense Strategies:

- **Online:** Rate-limiting, lockouts, progressive delays after failed attempts.
- **Offline:** Strong hashing (e.g., bcrypt, Argon2) with

salts, enforced password complexity, and minimal distribution of password hash files.

2. Biometric System Thresholds

- **Threshold Impact:**

- A *tighter* threshold lowers FAR (makes it harder for impostors) but raises FRR (legitimate users get rejected more).
- A *looser* threshold lowers FRR but raises FAR (more impostors may be accepted).

- **Security vs. Usability:**

- High threshold = safer from impersonation but more frustrated legitimate users.
- Low threshold = fewer user complaints but higher risk of unauthorized access.

3. Salting vs. Peppering

- **Salting:**

- Public, unique random value stored with each password hash.
- Defeats rainbow-table attacks by forcing each password to have a unique hash.

- **Peppering:**

- A *secret* salt not stored openly.
- Attackers, even with the hash file, can't easily validate guesses unless they also have the pepper.

- **Scenario:**

- If a database is stolen but the pepper is stored separately (e.g., on a secure hardware module), attackers must guess or locate that additional secret. This significantly slows down offline cracking.

4. Lamport Hash Chain

- **How It Works:**

1. Begin with a secret seed w .
2. Generate a chain of hashed values by repeatedly applying a one-way function H .
3. The server stores h_0 (final link in the chain). The user holds intermediate values.
4. Each login uses the *next* preimage in reverse order.

- **Advantage over SMS OTP:**

- SMS-based OTP can be intercepted or compromised via SIM swaps.

- Lamport chains rely on cryptographic one-way properties, do not need telecom infrastructure, and are fully under the user's control.

5. Password Composition Rules

- **Arguments For:**

- Force users to avoid overly simple passwords (e.g., "12345").
- Provide minimal complexity floors.

- **Arguments Against:**

- Users resort to predictable patterns (like "P@ssw0rd!").
- Usability suffers, leading to more password resets or reuse across sites.

- **Alternative:**

- Encourage passphrases (longer, easier to remember).
- Use deny lists of common passwords.
- Provide real-time feedback on password strength rather than fixed composition rules.

6. Account Recovery

- **Recovery Emails:**

- System sends a reset link or temporary code to an email address on file.
- Security depends on the email account itself being secure.

- **Secret Questions:**

- User answers pre-chosen questions to prove identity.
- Often guessable or available via public info (e.g., social media).

- **More Secure Approach:** Recovery emails are typically stronger, assuming the email account has robust security like MFA.

7. Rate-Limiting and Lockout Policies

- **Advantage of Strict Lockout:**

- Greatly reduces successful online brute forcing.

- **Drawback:**

- Legitimate users get locked out due to typos or minor mistakes.
- Potential for DoS attacks if attackers deliberately trigger lockouts.

- **Balanced Policy:**

- Temporary lockouts or progressive delays, combined with robust user identity checks if repeated fails occur.

8. Graphical Password Schemes

- Two Types:

1. **Click-based (cued recall)**: The user clicks points on an image in a correct sequence.
 - **Advantage**: More memory-friendly (spatial cues).
 - **Drawback**: Shoulder-surfing if someone observes the clicks.
2. **Android-style pattern locks (pure recall)**: The user traces a pattern on a grid.
 - **Advantage**: Faster for many users than typing.
 - **Drawback**: Smudge attacks on phone screens or easily guessable patterns.

9. Multi-Factor Authentication (MFA)

- **Definition**: Authentication using *two or more* from "Something you know" (password), "Something you have" (token), "Something you are" (biometric).
- **Security Improvement**: An attacker must compromise *all* factors, not just guess a password.
- **Potential Bypass**: Social engineering or phishing can still trick users into providing both password and token code if the system's implementation does not confirm origin authenticity.

10. Biometrics as Non-Secrets

- Why Non-Secret:

- Fingerprints, faces, iris patterns can be captured or photographed. People leave fingerprints on surfaces, faces can be filmed in public.

- Realistic Attack:

- High-resolution photo used to create a fake face or fingerprint mold.

- Mitigation:

- *Liveness* detection (e.g., checking blood flow, micro-movements, challenge responses).
- Combining biometrics with another authentication factor (2FA).

Part C

1. Password Space Calculation

- **(a)** Total possibilities: $\text{Space} = 62^{12}$
- 62 characters (A-Z, a-z, 0-9) raised to the 12th power.
- **(b)** Worst-case offline cracking time:
- Let $N = 62^{12}$ be the total passwords.
- Attacker speed: 10^8 guesses/s.
- Time in seconds: $\frac{N}{10^8}$.
- Convert to days: $\text{Days} = \frac{N}{(10^8 \times 86400)}$.

For a rough estimate:

$$62^{12} \approx 3.2 \times 10^{21} \quad \Rightarrow \quad \frac{3.2 \times 10^{21}}{10^8} = 3.2 \times 10^{13} \text{ seconds} \approx 3.2 \times 10^{13} / (8.64 \times 10^4) \approx 3.7 \times 10^8 \text{ days} \approx 10^6 \text{ years.}$$

1. Time to Crack with Iterated Hashing

- **Hash function:** 1 microsecond per hash.
- **(a)** With 10,000 iterations: Time per check = $10,000 \times 1\mu\text{s} = 10,000\mu\text{s} = 0.01 \text{ s}$ (10 ms).
- **(b)** For 10^9 password attempts: $10^9 \times 0.01 \text{ s} = 10^7 \text{ s} \approx 115.74 \text{ days}$

2. Biometric Error Rates

- **(a)** FRR of (2%): Out of 10,000 legitimate attempts, (2%) fail $\rightarrow 10,000 \times 0.02 = 200$ rejects.
- **(b)** FAR of (0.1%): Out of 10,000 impostor attempts, (0.1%) succeed $\rightarrow 10,000 \times 0.001 = 10$ false accepts.

3. One-Time Password Rate

- **Time-based OTP** changes every 30s:
 - The intercepted code becomes invalid after its 30s window. Replaying it at 45s will *fail*.
- **Reduction of Replay Attacks:**
 - Attackers have only a short window to reuse the same OTP.
 - Potential vulnerability: If the clock is out of sync or there's a short grace period, an attacker might slip in a brief replay. Typically, systems mitigate this by limiting each code to exactly one use.